

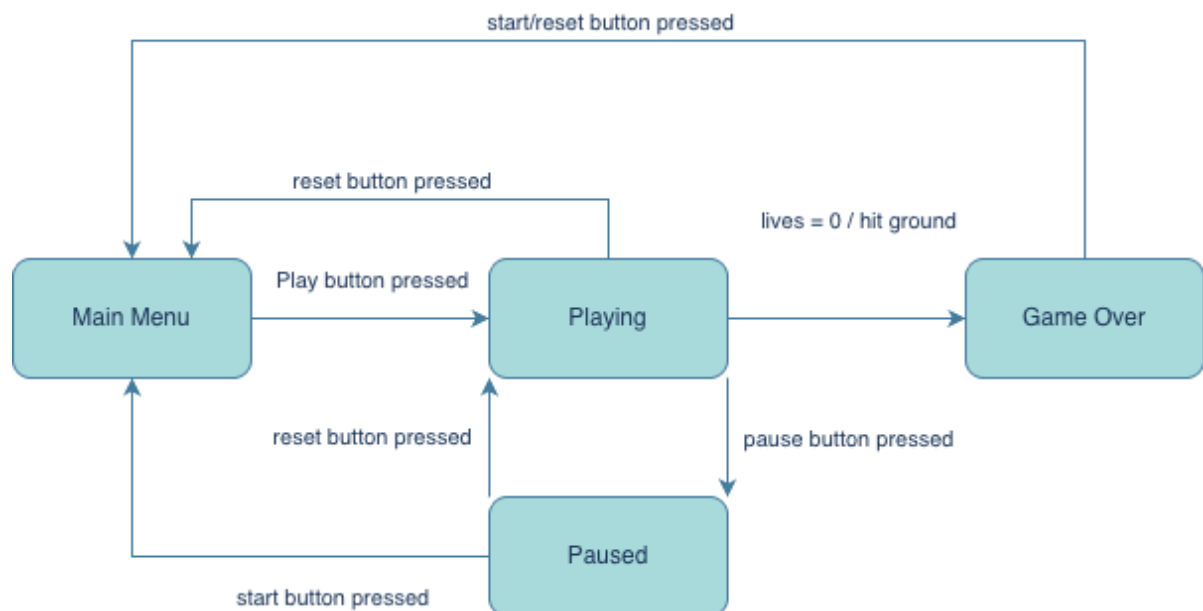
Game Strategy and Design Specifications

Our mini project is a simplified implementation of Flappy Bird on the DE0-CV FPGA board. The player controls a bird using a PS/2 mouse, while the game world scrolls from right to left on a 640 x 480 VGA display. The objective of the game is to keep the bird alive by avoiding obstacles and collecting gifts. The game uses the DE0-CV board as a standalone game console, while the VGA display is used for graphics, the PS/2 mouse controls movement, DIP switches are for mode selection and push buttons are used for starting, pausing, and resuming control whilst the seven segment displays shows score, lives, and the current level.

The game has two operation modes: Training mode and Single Player. Training mode allows the player to practice at the lowest difficulty level, this mode forces obstacle speeds to remain slow and constant. This mode will have a red background as well as the text "TRAINING" at the top. Single Player mode contains three levels, where difficulty increases as the player survives longer. This mode will have a green background with text at the top stating "GAME". Difficulty can be increased by raising the horizontal scrolling speed, increasing obstacle frequency, or reducing the size of the gap between obstacles.

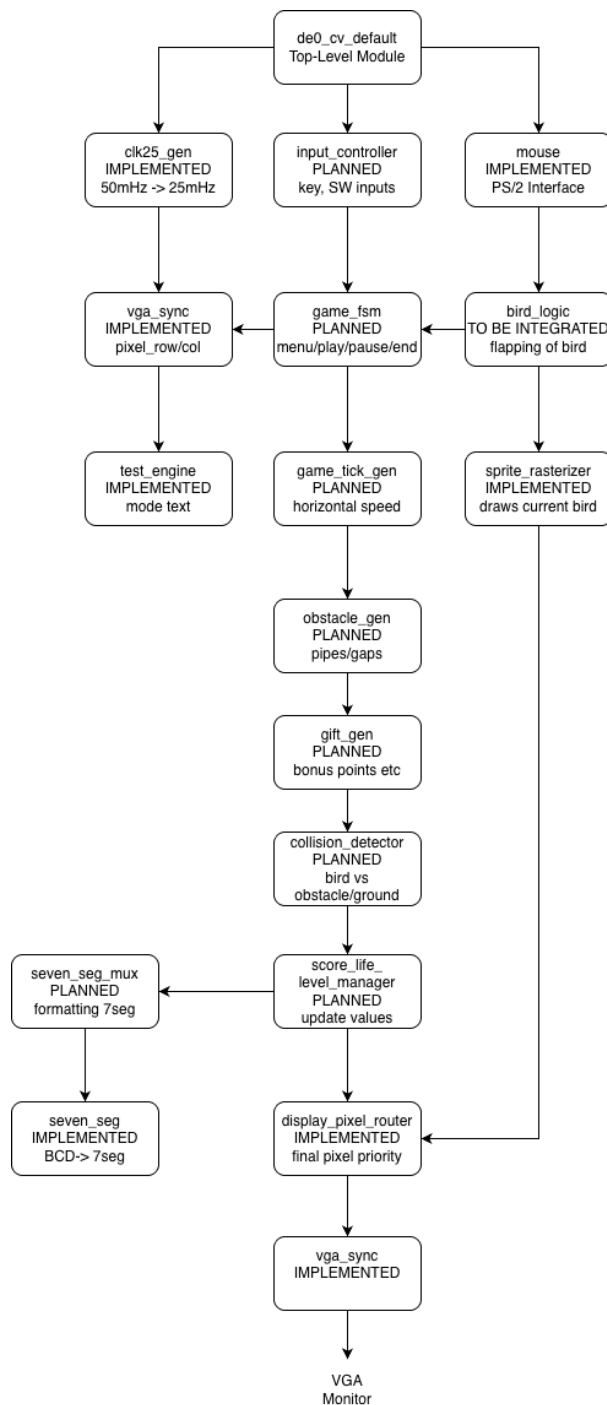
The bird moves vertically based upon PS/2 mouse input, left clicking causes the bird to flap upward, while the absence of input causes the bird to fall due to simulated gravity. The bird loses a life upon collision with an obstacle or the ground. When the life count reaches zero the game enters a game over state. The player gains points by passing obstacles or collecting gifts.

Game State Machine:



There are four main user visible states: Main Menu, Playing, Paused and Game Over. Upon power up or reset, the system enters the Main Menu state, where the player can select the game mode based on the DIP switches. To begin the game, the user must press the play push button, which then transitions to the Playing state, using the game configurations set within Main Menu. This is where the main game occurs, displaying the bird and obstacles, as well as the score and lives on the seven seg. If the pause button is pressed, then the FSM enters the paused state, where game updates are temporarily stopped, and the current visual screen is maintained. Pressing the pause button within the Paused state will return the user to the playing state where visuals resume, and the games logic continues. From the playing or paused state, the reset button can return the user to Main Menu, allowing the player to restart smoothly. If the bird's lives reach 0, or the bird has touched the ground, the FSM transitions to the Game Over state, displaying the score the user attained. From Game Over, pressing the start button or the reset button will return the user to the Main Menu state, where they can choose to reconfigure the game, or press the play button again and play with the same settings.

Block Diagram and Component Interfaces:



The block diagram shows the relationship between the currently implemented interface modules and the planned game specific modules. The top-level entity, **de0_cv_default** connects the DE0-CV board inputs and outputs to the internal game system. The **clk25_gen** module converts the 50MHz board clock into a 25MHz clock used by the VGA and mouse-related logic. The mouse module receives PS/2 mouse input and provides button signals which will be used to control the bird. The **vga_sync** module generates the VGA timing signals, providing the current **pixel_row** and **pixel_col**, used by the display modules: **test_engine**, **char_rom**, **sprite_rasterizer**, and **display_pixel_router**, combining mode text, the bird sprite and background colour before sending the final RGB signals to VGA.

The planned modules complete the game logic. **Game_fsm** controls Main Menu, Playing, Paused, and Game Over states. **Input_controller** handles switches toggling game mode, and the push-buttons for start, reset, and pause. Further modules handle movement, scrolling, collisions, scoring, lives, levels, and difficulty.